

OpsCore Secure Data Access Exercise

Graded Exercise — Student Packet · PostgreSQL

Learning Objectives

By completing this exercise, you will be able to:

- Identify SQL injection entry points (value slots vs. identifier slots) in a working PL/pgSQL function, and demonstrate a working exploit against each.
- Rewrite vulnerable dynamic SQL as parameterized functions, using bound values, array-based IN lists, and an allow-list for any identifier that cannot be bound.
- Prove that a fix actually works by re-running the original exploit and confirming it is now inert.

Setup

Run these two files, in order, in a scratch database (e.g. opscore_lab). Do not run them against any shared or production database.

- 00_opscore_schema.sql — the shared OpsCore schema (hosts, services, app_users, events, tickets). Provided by your instructor.
- opscore-exercise-starter.sql — adds a searchable summary column to tickets, seeds a few extra rows, and installs the two INSECURE functions you will audit in Part 1.

Do all of your Part 2 and Part 3 work inside the TODO blocks of opscore-exercise-starter.sql (or in a separate file you submit alongside it) — do not modify the two INSECURE functions themselves; you are auditing them as-is.

SUBMIT

(1) Your completed .sql file with all Part 2 and Part 3 code, and (2) this document with every "Your answer:" section filled in — pasted query output, exact payloads you used, and written responses.

Part 1 — Find the Injection

The two functions below are already installed by the starter file. Do not edit them — your job is to audit them, exploit them, and explain why each exploit works.

```
CREATE OR REPLACE FUNCTION usp_lookup_tickets_by_status_INSECURE(p_status TEXT)
```

```
RETURNS SETOF tickets LANGUAGE plpgsql AS $$
```

```
DECLARE v_sql TEXT;
```

```
BEGIN
```

```
v_sql := 'SELECT * FROM tickets WHERE status = ' || p_status || '';
```

```
RAISE NOTICE 'Assembled statement: %', v_sql;
```

```
RETURN QUERY EXECUTE v_sql;
```

```
END; $$;
```

```
CREATE OR REPLACE FUNCTION usp_search_tickets_INSECURE(p_keyword TEXT, p_sort_col TEXT)
```

```
RETURNS TABLE(ticket_id INT, kind TEXT, status TEXT, priority TEXT, summary TEXT, opened_at TEXT)
```

```
LANGUAGE plpgsql AS $$
```

```
DECLARE v_sql TEXT;
```

```
BEGIN
```

```
v_sql := 'SELECT ticket_id, kind::text, status::text, priority::text, summary::text, opened_at::text '
```

```
|| 'FROM tickets WHERE summary ILIKE '%' || p_keyword || '%' '
```

```
|| 'ORDER BY ' || p_sort_col;
```

```
RAISE NOTICE 'Assembled statement: %', v_sql;
```

```
RETURN QUERY EXECUTE v_sql;
```

END; \$\$;

1.1 Classify the entry points

For each snippet below, write whether the highlighted input lands in a VALUE slot or an IDENTIFIER slot, and whether the snippet as shown is vulnerable or already safe. One sentence of justification each.

- (a) ...WHERE priority = "" || p_priority || ""
- (b) ...ORDER BY ' || p_column
- (c) ...WHERE priority = p_priority (p_priority is a function parameter, no concatenation)
- (d) ...WHERE summary ILIKE '%' || p_keyword || '%'

Your answer:

1.2 Exploit the tautology

Craft a single call to `usp_lookup_tickets_by_status_INSECURE(...)` that returns every ticket in the table regardless of its actual status. Run it, then paste (a) the assembled statement from the RAISE NOTICE line, (b) the number of rows returned, and (c) a 1–2 sentence explanation of why the WHERE clause no longer filters anything.

Your answer:

1.3 Exploit the UNION-based read

Craft a call to `usp_search_tickets_INSECURE(...)` whose `p_keyword` value closes the ILIKE pattern and appends a UNION SELECT that pulls username and email out of `app_users`. Paste the assembled statement and the result rows, and state exactly which two real columns were exfiltrated and which positions the NULL placeholders had to fill.

Your answer:

Part 2 — Parameterize

Write each of the following in your .sql file. None of them should build a query string by concatenating a value into it.

2.1 usp_lookup_tickets_by_status(p_status TEXT)

A secured replacement for artifact A. This one needs no dynamic SQL at all — a plain parameterized query is enough.

2.2 usp_search_tickets(p_keyword TEXT, p_sort_key TEXT)

- Bind the keyword as a value — never concatenate it into the statement text.
- `p_sort_key` is structure, not a value: allow-list it to one of 'opened_at', 'priority', 'status' with a CASE expression; anything else should default to 'opened_at' rather than erroring.
- Use `format()` with the %I specifier to compose the validated identifier into the ORDER BY clause.

2.3 tickets_by_priority(p_priorities TEXT[])

Accept an array of priority codes (e.g. `ARRAY['p1','p2']`) and return every ticket whose priority is in that array, using `= ANY(...)`.

2.4 Prove the fix

Re-run your exact payloads from 1.2 and 1.3 against the two new secured functions. Paste both results and state, in one sentence each, why each payload is now inert.

Your answer:

Part 3 — Reflection

Answer each in 2–4 sentences, in your own words.

3.1 Why can a bound parameter never become code?

Explain why a value bound through a parameter — even one containing quotes or SQL keywords — can never change the structure of the query it's bound into.

Your answer:

3.2 Why isn't the sort column "just another parameter"?

In `usp_search_tickets`, the keyword is bound directly, but the sort column goes through an allow-list instead of a parameter. Explain what would go wrong if you tried to bind `p_sort_key` directly as a parameter to ORDER BY, and why concatenating it raw (without the allow-list) would be dangerous.

Your answer:

3.3 What would undo your Part 3 protections?

Suppose `helpdesk_app` were accidentally also granted PostgreSQL's built-in `pg_read_all_data` role. What would that undo, and why does the view you built stop being an effective boundary in that scenario?

Your answer:

3.4 Why doesn't "escape the quotes" work as a defense?

Give one concrete situation (from this exercise or the lessons) where escaping quotes or blocking specific keywords would fail to stop an injection that parameterization stops.

Your answer: